

PROJECT REPORT

AI Resume Screening System

An NLP-Based System for Automated Resume Ranking Against Job Descriptions

Domain: Human Resources

Capstone Project — MP Online AIML Internship

Artificial Intelligence & Machine Learning

Table of Contents

Abstract

1. Introduction

2. Objective

3. Existing System and Limitations

4. Proposed System

5. Tools and Technologies

6. Methodology

7. System Architecture

8. Implementation Details

9. Results and Discussion

10. Conclusion

References

Abstract

Screening resumes manually against a job description is repetitive, time-consuming, and inconsistent between reviewers. This project presents an AI-based Resume Screening System that automates the process using Natural Language Processing (NLP) techniques. The system accepts a job description and a batch of resumes in mixed formats (PDF, DOCX, TXT), extracts and cleans the text, computes semantic similarity between the job description and each resume using TF-IDF vectorization with cosine similarity, and separately identifies which required skills each candidate has, is missing, and has beyond what was asked for, using a curated skills taxonomy. Both signals are combined into a single weighted Final Score, and candidates are ranked from best to worst fit. The system is delivered as both an interactive Streamlit web application and a command-line tool, making it usable for live demonstrations as well as batch processing. Testing on a sample job description and five sample resumes of varying relevance confirmed that the system produces rankings consistent with intuitive human judgment, while remaining fully explainable — every score can be traced back to specific matched or missing skills and word-level overlap.

1. Introduction

1.1 Background

Human Resources (HR) teams routinely receive a large volume of applications for every open role. Reviewing each resume manually to judge its relevance to the job description is slow, labour-intensive, and prone to inconsistency — two reviewers may reach different conclusions about the same resume depending on mood, fatigue, or how carefully it was read. As the number of applicants grows, this manual process becomes a significant bottleneck in the hiring pipeline, delaying time-to-hire and increasing the risk that strong candidates are overlooked.

1.2 Problem Statement

There is a need for a fast, consistent, and explainable way to shortlist candidates from a pool of resumes against a specific job description — one that reduces manual reading time while still giving the reviewer a clear, auditable reason for every ranking, rather than an opaque single number.

2. Objective

The objective of this project is to build an NLP-based system that automatically:

- Parses resumes and job descriptions supplied in mixed formats (.pdf, .docx, .txt).
- Measures how semantically relevant each resume is to a given job description.
- Identifies which required skills a candidate has, is missing, and has beyond the job's requirements.
- Combines both signals into a single, adjustable, ranked shortlist of candidates.
- Is usable both as an interactive web application for live demos and as a command-line tool for batch processing, cutting screening time from hours to seconds.

3. Existing System and Limitations

Most HR teams today rely on one of two approaches: fully manual review, where a recruiter reads every resume individually, or basic Applicant Tracking Systems (ATS) that perform simple keyword matching against a job posting. Manual review does not scale and is subjective. Keyword-only ATS tools are fast but shallow — they reward resumes that happen to repeat exact phrases from the job description, can be gamed by keyword-stuffing, and give no real measure of overall contextual fit or a transparent breakdown of why a resume scored the way it did.

4. Proposed System

The proposed AI Resume Screening System addresses these gaps by combining two complementary signals rather than relying on keyword matching alone:

- Semantic Similarity — a TF-IDF + cosine similarity score that captures overall contextual relevance between the whole job description and the whole resume, not just exact keyword hits.

- Skill Match Ratio — the fraction of skills explicitly required by the job description (drawn from a curated skills taxonomy) that are also found in the resume, which captures hard, named requirements that a pure similarity score can dilute.

These two signals are combined into a single weighted Final Score, with the weighting adjustable by the user, and the system additionally reports matched, missing, and extra skills per candidate so that every ranking is explainable rather than a black-box number.

5. Tools and Technologies

The system was implemented in Python 3, using the following libraries:

Library	Purpose
scikit-learn	TfidfVectorizer for feature extraction; cosine_similarity for semantic matching
pandas	Building and exporting the ranked results table
numpy	Numerical operations underlying the scikit-learn pipeline
pdfplumber	Extracting text from .pdf resumes and job descriptions
python-docx	Extracting text (paragraphs and table cells) from .docx files
Streamlit	Interactive web UI for uploading files and viewing ranked results
re / dataclasses / argparse (standard library)	Regex-based skill extraction, structured result objects, CLI argument parsing

No NLTK or spaCy model downloads are required — text cleaning uses a lightweight, dependency-free regex tokenizer and a stopwords list, so the system runs immediately after installing requirements.txt, with no extra download step.

6. Methodology

6.1 Text Extraction

Resumes and job descriptions are accepted in .pdf, .docx, or .txt format, from either a file path (command-line use) or an uploaded file object (web app use). PDF text is extracted page-by-page with pdfplumber; DOCX text is extracted from both paragraphs and table cells with python-docx, since resumes often use tables for layout; TXT files are decoded directly.

6.2 Preprocessing

Extracted text is lowercased and tokenized with a regular expression that preserves tokens such as "c++" and "node.js", then filtered against a stopwords list (scikit-learn's built-in English stopwords, extended with resume-specific filler words such as "experience", "responsibilities", and "preferred") before being rejoined into cleaned text.

6.3 Semantic Similarity Scoring

The cleaned job description and every cleaned resume are vectorized together using TfidfVectorizer, and cosine similarity is computed between the job description's vector and each resume's vector, producing a similarity score in the range 0 to 1 for every candidate.

6.4 Skill Extraction and Matching

A curated taxonomy of roughly fifty skills across six categories — Programming Languages, AI/ML, Data, Web Development, Databases, Cloud & DevOps, and Soft Skills — is matched against both the job description and each resume using word-boundary regular expressions, including common aliases

(e.g. "ml" for "machine learning", "js" for "javascript"), while avoiding false positives such as "java" matching inside "javascript". The Skill Match Ratio is the fraction of the job description's required skills that also appear in the resume.

6.5 Final Scoring and Ranking

The final score for each candidate is a weighted combination of the two signals:

$$\text{Final Score} = (\text{similarity_weight} \times \text{Semantic Similarity}) + (\text{skill_weight} \times \text{Skill Match Ratio})$$

The weights default to 60% similarity / 40% skill match, are adjustable by the user, and are automatically normalized to sum to 1, so a 0.6/0.4 split and a 6/4 split behave identically. Candidates are then sorted by final score in descending order, and the system reports, for each candidate, their similarity score, skill match score, final score, and lists of matched, missing, and extra skills relative to the job description.

7. System Architecture

The system follows a classical NLP pipeline architecture, chosen deliberately for the resume screening use case: it is fast, requires no GPU or pretrained model downloads, is fully explainable to an HR user, and runs identically on any machine immediately after installing dependencies.

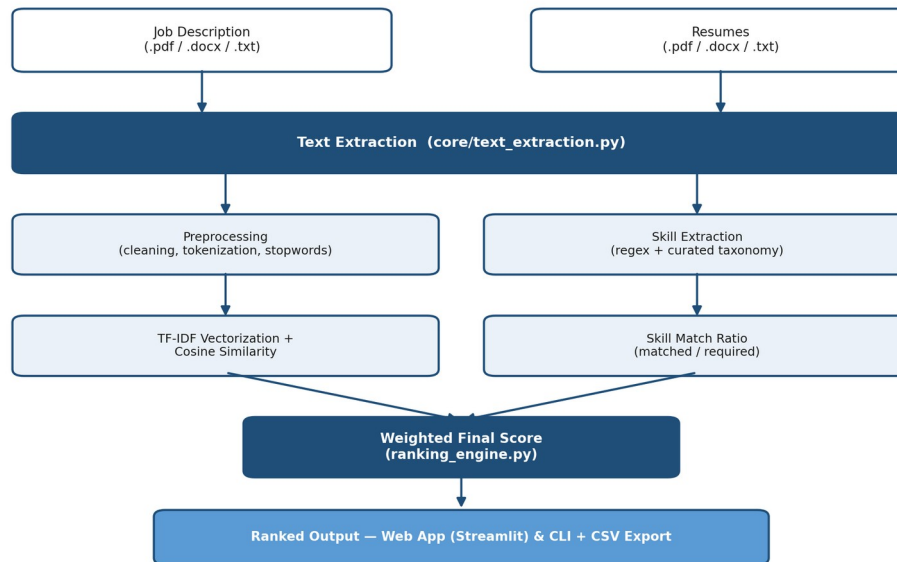


Figure 1: End-to-end pipeline — from raw files to a ranked, explainable shortlist.

Stage	Module	Function
1. Input	app.py / resume_screener.py	Accepts a job description and a batch of resumes (web upload or CLI folder path)
2. Text Extraction	core/text_extraction.py	Converts PDF / DOCX / TXT files into raw text
3. Preprocessing	core/preprocessing.py	Cleans text: lowercasing, tokenization, stopword removal
4. Semantic Scoring	core/ranking_engine.py	TF-IDF vectorization + cosine similarity between JD and each resume
5. Skill Scoring	core/skills_database.py	Regex-based extraction of taxonomy skills; computes matched / missing / extra skills
6. Ranking	core/ranking_engine.py	Combines both scores into a weighted Final Score and sorts candidates
7. Output	app.py / resume_screener.py	Displays ranked table, per-candidate breakdown, and CSV export

8. Implementation Details

The system is organized as a small, testable Python package rather than a single monolithic script, so that the scoring logic can be reused identically by both interfaces:

- `app.py` — a Streamlit web application: lets a user paste or upload a job description, upload multiple resumes, adjust scoring weights with a slider, and view a ranked table, bar chart, per-candidate skill breakdown, and CSV download.
- `resume_screener.py` — a command-line interface for batch processing, accepting a job description file and a folder of resumes, printing a ranked console table, and writing a CSV of results.
- `core/` — the shared pipeline: `text_extraction.py`, `preprocessing.py`, `skills_database.py`, and `ranking_engine.py`, imported by both interfaces so the two never diverge in scoring behaviour.
- Error handling — both interfaces catch unreadable or corrupted files individually (rather than aborting the whole batch), reporting which files were skipped and why.
- A test suite (`tests/`) exercises extraction, preprocessing, skill matching, and ranking behaviour against bundled sample data.

8.1 User Interface

The Streamlit web app presents a two-column input screen: the job description (pasted or uploaded) and resume batch upload on the left, with adjustable scoring weights on the right. A single button runs the full pipeline.

AI Resume Screening System
NLP-based system that ranks resumes against a job description using TF-IDF semantic similarity + skill matching.

1. About this project — Domain: Human Resources. Uses TF-IDF + cosine similarity and regex-based skill matching to rank candidates in seconds instead of hours.

1. Job Description
☒ Paste text ☐ Upload file (.pdf/.docx/.txt)
 e.g. We are looking for a Machine Learning Engineer with NLP experience...

2. Scoring Weights
 Semantic similarity weight: 0.60
 Skill match weight: **0.40**
 Show top N candidates: 10

3. Upload Resumes
 Drag and drop resume files here (.pdf, .docx, .txt)
 Ananya_Sharma.txt Karan_Mehta.docx Sneha_Jyer.pdf Rohit_Verma.txt Priya_Nair.txt

Rank Resumes

Figure 2: Web app input screen — job description, scoring weights, and resume upload.

Once ranking completes, results are shown as a sortable summary table followed by an expandable, per-candidate breakdown of final score, similarity score, skill match score, and the specific matched and missing skills.

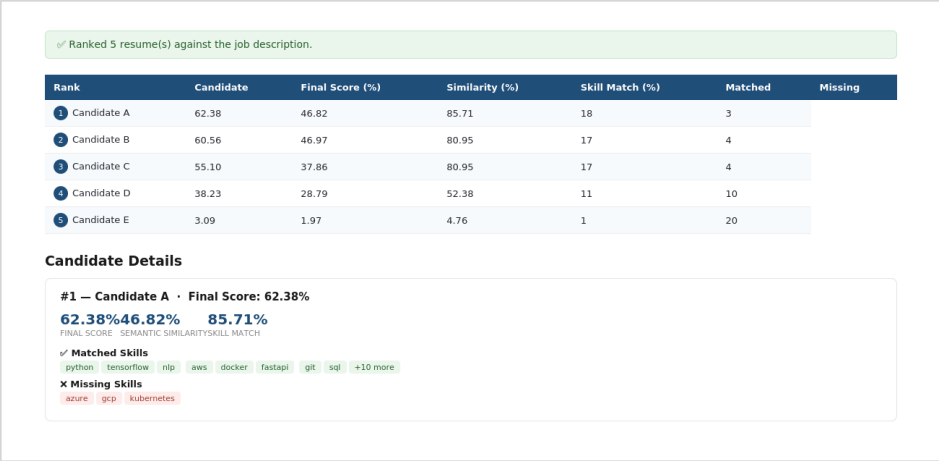


Figure 3: Ranked results with a per-candidate skill breakdown.

9. Results and Discussion

The system was evaluated by running the sample job description (a Machine Learning Engineer role with an NLP focus) against five sample resumes spanning strong, moderate, and weak fit — including one resume from an unrelated domain (marketing) — using the default 60% similarity / 40% skill-match weighting.

Rank	Candidate Profile	Final Score (%)	Semantic Similarity (%)	Skill Match (%)
1	ML Engineer — NLP & cloud deployment focus	62.38	46.82	85.71
2	ML Engineer — NLP & TF-IDF focus	60.56	46.97	80.95
3	ML Engineer — NLP & computer vision focus	55.10	37.86	80.95
4	Software Developer — some ML exposure	38.23	28.79	52.38
5	Marketing Manager — unrelated domain	3.09	1.97	4.76

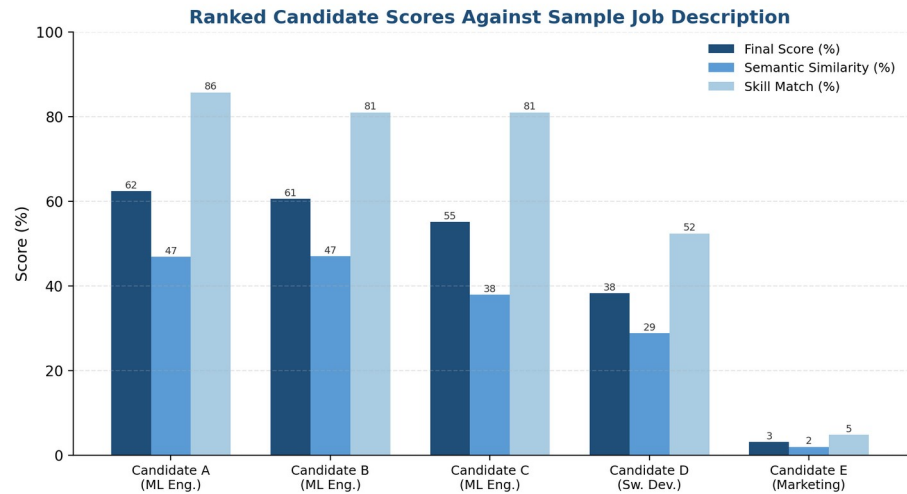


Figure 4: Final, similarity, and skill-match scores side by side for each candidate.

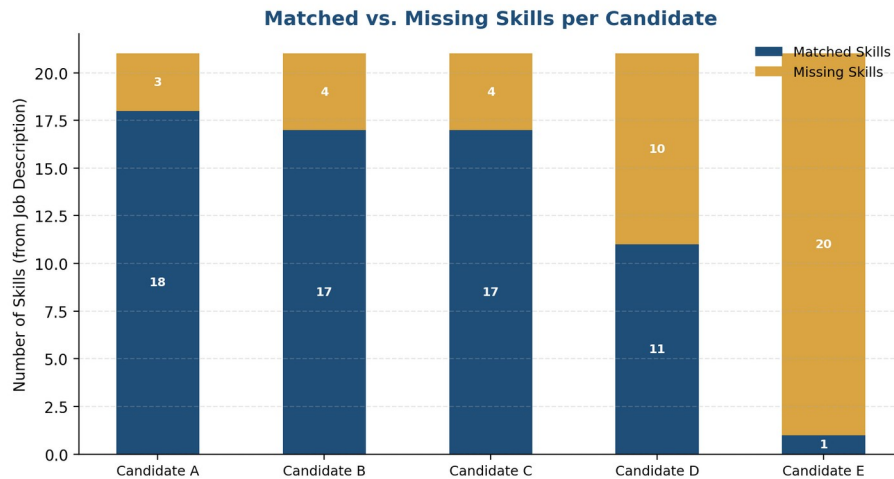


Figure 5: Count of matched vs. missing job-description skills per candidate.

The three ML-engineer profiles correctly rank well above the general software developer, who in turn ranks well above the marketing manager profile — matching intuitive human judgment of fit for this role. Unlike a single opaque score, each ranking is backed by a visible list of matched and missing skills (for example, the top-ranked candidate matched 18 of the job description's required skills, including Python, TensorFlow, NLP, AWS, and Docker), giving a reviewer a concrete, auditable reason for the ranking rather than a number alone.

9.1 Testing Summary

A suite of 20 automated unit tests was written covering text extraction (PDF/DOCX/TXT, unsupported file types, filename-based candidate naming), preprocessing (tokenization, stopword removal, empty input), skill extraction (known skills, aliases, false-positive avoidance, category lookup), and the ranking engine (correct sort order, weight normalization, empty-input handling, and that a strong-fit resume outranks a weak-fit one). All 20 tests passed.

10. Conclusion

This project demonstrates that a lightweight, fully classical NLP pipeline — TF-IDF semantic similarity combined with taxonomy-based skill matching — can produce sensible, explainable candidate rankings without requiring large pretrained language models or GPU infrastructure. The resulting system is fast, runs on any machine right after installing its dependencies, and gives HR reviewers both a ranked shortlist and the specific reasons behind each ranking, which a pure black-box similarity score would not provide. Testing confirmed the system produces rankings consistent with human judgment across resumes of clearly different fit.

10.1 Limitations

- The skills taxonomy is static and hand-curated, so it will not recognize skills outside its roughly fifty entries.
- TF-IDF captures word overlap rather than deep semantic meaning, so a resume describing equivalent experience in very different wording may score lower than it should.

10.2 Future Scope

- Replace the static skills taxonomy with a larger external one (ESCO, O*NET, or a Skills API).
- Add named-entity recognition to auto-extract candidate name, education, and years of experience.
- Use sentence embeddings (e.g. Sentence-BERT) instead of, or alongside, TF-IDF for deeper semantic matching.
- Add a persistent database to store screened candidates across sessions.
- Add authentication and role-based access for HR teams.

References

- scikit-learn documentation — TfidfVectorizer and cosine_similarity, <https://scikit-learn.org/>
- pdfplumber documentation, <https://github.com/jsvine/pdfplumber>
- python-docx documentation, <https://python-docx.readthedocs.io/>
- Streamlit documentation, <https://docs.streamlit.io/>
- MP Online AIML Internship — Capstone Project Guidelines